

The word *argument* is a bit confusing. Generally speaking, it refers to anything that the command addresses, so anything you put after the command is an argument (including the options). Apart from the options that can be used as an argument, commands can have other arguments as well, which serve as a target to the command.

Let’s have a look at an example: the command **ls -l /etc**. This command has two different arguments: **-l** and **/etc**. The first argument is an option, modifying the behavior of the command. The second argument is a target, specifying where the command should do its work. You’ll find these three elements in nearly all commands you work with in a Linux environment.

Executing Commands

The purpose of the Linux shell is to provide an environment in which commands can be executed. The shell takes care of interpreting the command that a user has entered correctly. To do this, the shell makes a distinction between three kinds of commands:

- Aliases
- Internal commands
- External commands

An *alias* is a command that a user can define as needed. Some aliases are provided by default; type **alias** on the command line to get an overview. To define an alias, use **alias newcommand=‘oldcommand’** (as in the default alias **ll=‘ls -l --color=auto’** that has already been created on your system). Aliases are executed before anything else. So, if you have an alias with the name **ll** but also a command with the name **ll**, the alias will always take precedence for the command, unless a complete pathname like `/usr/bin/ls` is used.

An *internal command* is a command that is a part of the shell itself and, as such, doesn’t have to be loaded from disk separately. An *external command* is a command that exists as an executable file on the disk of the computer. Because it has to be read from disk the first time it is used, it is a bit slower. When a user executes a command, the shell first looks to determine whether it is an internal command; if it is not, it looks for an executable file with a name that matches the command on disk. To find out whether a command is a Bash internal command or an executable file on disk, you can use the **type** command. Use for instance **type pwd** to find out that the **pwd** command that will be executed is really an alias.

To change how external commands are found by the shell, use the *\$PATH* variable. This variable defines a list of directories that is searched for a matching filename when a user enters a command. To find out which exact command the shell will be using, you can use the **which** command. For instance, type **which ls** to find out where the shell will get the **ls** command from. An even stronger command is **type**, which will also work on internal commands and aliases.

You should notice that, for security reasons, the current directory is not in the *\$PATH* variable and Linux does not look in the current directory to see whether a specific command is available from that directory. That is why you need to start a command that is in the current directory but nowhere in the *\$PATH* by including `./` in front of it. The dot stands for the current directory, and by running it as `./`, you tell Bash to look for the command in the current directory. Although running commands this way is not very common, you will have to do it to run scripts that you’ve created in your current directory.

The *\$PATH* variable can be set for specific users, but in general, most users will be using the same *\$PATH* variable. The only exception to this is the user root, who needs access to specific administration commands. In [Exercise 2-1](#), you learn some of the basics about working with commands.

Exercise 2-1 Using Internal and External Commands from the Shell

1. Authenticate on the server1 server that you created in [Chapter 1](#), “[Installing Red Hat Enterprise Linux](#),” as the user that you also created in [Chapter 1](#) when installing your server.
2. Click **Activities**. In the Search bar that appears, type **term** and click the terminal icon that shows to open a terminal. All exercises in this book are intended to be executed in a terminal.
3. Type **time ls**. This executes the **ls** command where the Bash internal **time** shows information about the time it took to complete this command.
4. Type **which time**. This shows the filename `/usr/bin/time` that was found in the *\$PATH* variable.
5. Type **time**, which shows that **time** is a shell keyword.
6. Type **echo \$PATH** to show the contents of the *\$PATH* variable. You can see that `/usr/bin` is included in the list, but because there also is an internal command **time**, the **time** command from the path will not be executed unless you tell the shell specifically to do so—the command in step 3 has executed the internal command for you because of command precedence.
7. Type **/usr/bin/time ls** to run the **/usr/bin/time** command when executing **ls**. You’ll notice that the output differs completely. Ignore the meaning of the output; we get back to that later. What matters for now is that you realize that these are really two different commands.

I/O Redirection

By default, when a command is executed, it shows its results on the screen of the computer you are working on. The computer monitor is used as the standard destination for output, which is also referred to as STDOUT. The shell also has default standard destinations to send error messages to (STDERR) and to accept input (STDIN). [Table 2-2](#) gives an overview of all three.

Table 2-2 Standard Input, Output, and Error Overview

Name	Default Destination	Use in Redirection	File Descriptor Number
<i>STDIN</i>	Computer keyboard	< (same as 0<)	0

Name	Default Destination	Use in Redirection	File Descriptor Number
------	---------------------	--------------------	------------------------

<u>STDOUT</u>	Computer monitor	> (same as 1>)	1
----------------------	------------------	----------------	---

<u>STDERR</u>	Computer monitor	2>	2
----------------------	------------------	----	---

So if you run a command, that command would expect input from the keyboard, and it would normally send its output to the monitor of your computer without making a distinction between normal output and errors. Some commands, however, are started in the background and not from a current terminal session, so these commands do not have a monitor or console session to send their output to, and they do not listen to keyboard input to accept their standard input. That is where redirection comes in handy. **Redirection** is also useful if you want to work with input from an alternative location, such as a file.

Programs started from the command line have no idea what they are reading from or writing to. They just read from what the Linux kernel calls file descriptor 0 if they want to read from standard input, and they write to file descriptor number 1 to display non-error output (also known as “standard output”) and to file descriptor 2 if they have error messages to be output. By default, these **file descriptors** are connected to the keyboard and the screen. If you use redirection symbols such as <, >, and |, the shell connects the file descriptors to files or other commands. Let’s first look at the redirectors < and >. Later we discuss pipes (the | symbol). **Table 2-3** shows the most common redirectors that are used from the Bash shell.

Table 2-3 Common Bash Redirectors

Redirector	Explanation
> (same as 1>)	Redirects STDOUT. If redirection is to a file, the current contents of that file are overwritten.
>> (same as 1>>)	Redirects STDOUT in append mode. If output is written to a file, the output is appended to that file.
2>	Redirects STDERR.
2>&1	Redirects STDERR to the same destination as STDOUT. Notice that this has to be used in combination with normal output redirection, as in ls whuhiu > errout 2>&1 .

< (same as 0<)	Redirects STDIN.
----------------	------------------

In I/O redirection, files can be used to replace the default STDIN, STDOUT, and STDERR. You can also redirect to **device files**. A device file on Linux is a file that is used to access specific hardware. Your hard disk, for instance, can be referred to as /dev/sda in most cases, the console of your server is known as /dev/console or /dev/tty1, and if you want to discard a command’s output, you can redirect to /dev/null. Note that to access most device files, you need to have root privileges.

Using Pipes

Whereas an I/O redirector is used as an alternative for a keyboard and computer monitor, a **pipe** can be used to catch the output of one command and use that as input for a second command. If a user runs the command **ls**, for instance, the output of the command is shown onscreen, because the screen is the default STDOUT. If the user uses **ls | less**, the commands **ls** and **less** are started in parallel. The standard output of the **ls** command is connected to the standard input of **less**. Everything that **ls** writes to the standard output will become available for reading from standard input in **less**. The result is that the output of **ls** is shown in the **less** pager, where the user can browse up and down through the results easily.

As a Linux administrator, you will use pipes a lot. Using pipes makes Linux a flexible operating system; by combining multiple commands using pipes, you can create “super” commands that make almost anything possible. In **Exercise 2-2**, you use I/O redirectors and pipes.

Exercise 2-2 Using I/O Redirection and Pipes

1. Open a shell as user **student** and type **cd** without any arguments. This ensures that the home directory of this user is the current directory while working on this exercise. Type **pwd** to verify this.

2. Type **ls**. You’ll see the **ls** command output onscreen.

3. Type **ls > /dev/null**. This redirects STDOUT to the null device, with the result that you will not see it.

4. Type **ls ilwehgi > /dev/null**. This command shows a “no such file or directory” message onscreen. You see the message because it is not STDOUT, but rather an error message that is written to STDERR.

5. Type **ls ilwehgi 2> /dev/null**. Now you will no longer see the error message.

6. Type **ls ilwehgi /etc 2> /dev/null**. This shows the contents of the /etc folder while hiding the error message.

7. Type **ls ilwehgi /etc 2> /dev/null > output**. In this command, you still write the error message to /dev/null while sending STDOUT to a file with the name output that will be created in your home directory.
8. Type **cat output** to show the contents of this file.
9. Type **echo hello > output**. This overwrites the contents of the output file. Verify this by using **cat output** again.
10. Type **ls >> output**. This appends the result of the **ls** command to the output file. Type **cat output** to verify.
11. Type **ls -R /**. This shows a long list of files and folders scrolling over your computer monitor. (You might want to press Ctrl-C to stop [or wait some time]).
12. Type **ls -R /.** | **less**. This shows the same result, but in the **less** pager, where you can scroll up and down using the arrow keys on your keyboard.
13. Type **q** to close **less**. This will also end the **ls** program.
14. Type **ls > /dev/tty1**. This gives an error message because you are executing the command as an ordinary user, and ordinary users cannot address device files directly (unless you were logged in to tty1). Only the user root has permission to write to device files directly.

History

A convenient feature of the Bash shell is the Bash history. Bash is configured by default to keep the last 1,000 commands a user used. When a shell session is closed, the history of that session is updated to the history file. The name of this file is .bash_history and it is created in the home directory of the user who started a specific shell session. Notice that the history file is written to only when the shell session is closed; until that moment, all commands in the history are kept in memory.

The history feature makes it easy to repeat complex commands. There are several ways of working with history:

- Type **history** to show a list of all commands in the Bash history.
- Press Ctrl-r to open the prompt from which you can do backward searches in commands that you have previously used. Just type a string and Bash will look backward in the command history for any command containing that string as the command name or one of its arguments. Press Ctrl-r again to repeat the last backward search.
- Type **!number** to execute a command with a specific number from history.
- Use **history -d number** to delete a specific command from history. Notice that this command will renumber all other lines in history: if you’ve removed line 31, the line previously numbered as line 32 will now be line 31.
- Type **!sometext** to execute the last command that starts with *sometext*. Notice that this is a potentially dangerous command because the command that was found is executed immediately!

In some cases it might be necessary to wipe the Bash history. This capability is useful, for instance, if you’ve typed a password in clear text by accident. If that happens, you can type **history -c** to clear the current history. Commands from this session won’t be written to the history file when you exit the current session. If you want to remove both the current history and the contents of the .bash_history file, then type **history -w** immediately after running the **history -c** command. Alternatively, use **history -d number** to remove a specific command from history.

[Exercise 2-3](#) guides you through some history features.

Exercise 2-3 Working with History

1. Make sure that you have opened a shell as user **student**.
2. Type **history** to get an overview of commands that you have previously used.
3. Type some commands, such as the following:

```
ls
pwd
cat /etc/hosts
ls -l
```

The goal is to fill the history a bit.
4. Open a second terminal on your server. To do so, click **Activities** in the upper-left corner, and in the Search bar, type **term**. Next, click the terminal window to start it.
5. Type **history** from this second terminal window. Notice that you do not see the commands that you just typed in the other terminal. The reason is that the history file has not been updated yet.
6. From the first terminal session, press Ctrl-r. From the prompt that opens now, type **ls**. You’ll see the last **ls** command you used. Press Ctrl-r again. You’ll now see that you are looking backward and that the previous **ls** command is highlighted. Press Enter to execute it.
7. Type **history | grep cat**. The **grep** command searches the history output for any commands that contain the text *cat*. Note the command number of one of the **cat** commands you have previously used.
8. Type **!nn**, where *nn* is replaced by the number you noted in step 7. You’ll see that the last **cat** command is repeated.
9. Close this terminal by typing **exit**.
10. From the remaining window, type **history -c**. This wipes all history that is currently in memory. Close this terminal session as well.
11. Open a new terminal session and type **history**. The result may be a bit unexpected, but you’ll see a list of commands anyway. The reason is that **history -c** clears the in-memory history, but it does not remove the .bash_history file in your home directory.

Bash Completion

Another useful feature of the Bash shell is command-line completion. This feature helps you to find the command that you need, and it also works on variables and filenames.

Bash completion is useful when you’re working with commands. Just type the beginning of a command and press the Tab key. If there is only one option for completion, Bash will complete the command automatically for you. If there are several options, you need to press Tab once more to get an overview of all the available